

Prédiction de la consommation énergétique d'un drone UAV

Plan du notebook

1. Introduction
2. Chargement des données et description du dataset
3. Exploration et préparation des données
4. Définition du problème d'apprentissage
5. Entraînement et comparaison de modèles
6. Conclusion

1.Introduction

Dans ce projet, nous cherchons à analyser et modéliser la consommation énergétique d'un drone quadricoptère à partir de données réelles de vol.

Le dataset contient des mesures collectées lors de plusieurs vols, incluant des informations sur les conditions environnementales, la dynamique du drone et sa consommation électrique.

L'objectif principal est de construire un modèle capable de prédire la consommation énergétique en fonction des paramètres de vol.

2.Chargement des données et description du dataset

Dans cette étape, nous chargeons les datasets contenant les informations de vol et les paramètres associés.

Le dataset est composé de deux fichiers :

- `flights.csv` : données temporelles enregistrées pendant les vols
- `parameters.csv` : paramètres globaux des vols.

```
In [1]: import pandas as pd
```

```
In [2]: flights=pd.read_csv('flights.csv')
```

```
C:\Users\DEM\AppData\Local\Temp\ipykernel_22072\2302544006.py:1: DtypeWarning: Columns (24) have mixed types. Specify dtype option on import or set low_memory=False.  
flights=pd.read_csv('flights.csv')
```

In [3]: `flights`

Out[3]:

	flight	time	wind_speed	wind_angle	battery_voltage	battery_c
0	1	0.00	0.1	12	24.222174	0.0
1	1	0.20	0.1	3	24.227180	0.0
2	1	0.30	0.1	352	24.225929	0.0
3	1	0.50	0.1	354	24.224678	0.0
4	1	0.60	0.1	359	24.210905	0.0
...	...	...	...	...	...	...
257891	279	152.10	1.1	198	22.857437	0.0
257892	279	152.20	1.1	196	22.847422	0.0
257893	279	152.41	1.2	189	22.856186	0.1
257894	279	152.60	1.1	187	22.854933	0.1
257895	279	152.70	1.1	184	22.861195	0.1

257896 rows × 28 columns

In [4]: `#Taille
flights.shape`

Out[4]: (257896, 28)

In [5]: `# Aperçu des 3 premières lignes données de vol
flights.head(3)`

Out[5]:

	flight	time	wind_speed	wind_angle	battery_voltage	battery_current
0	1	0.0	0.1	12	24.222174	0.087470
1	1	0.2	0.1	3	24.227180	0.095421
2	1	0.3	0.1	352	24.225929	0.095421

3 rows × 28 columns

In [6]: `parameters=pd.read_csv('parameters.csv')`

In [7]: `parameters`

Out[7]:

	flight	speed	payload	altitude	date	local_time	route
0	1	4	0	25	4/7/2019	10:13	R5
1	2	4	0	50	4/7/2019	10:23	R5
2	3	6	0	25	4/7/2019	10:33	R5
3	4	8	0	25	4/7/2019	10:48	R5
4	5	4	0	25	4/7/2019	11:05	R2
...	...	...	...	...	...	...	...
204	275	8	500	25	10/24/2019	9:05	R1
205	276	10	500	25	10/24/2019	9:32	R1
206	277	10	500	25	10/24/2019	9:45	R1
207	278	10	500	25-50-100-25	10/24/2019	10:00	R7
208	279	10	0	25-50-100-25	10/24/2019	10:10	R7

209 rows x 7 columns

```
In [8]: # Aperçu des données de vol
parameters.head(3)
```

Out[8]:

	flight	speed	payload	altitude	date	local_time	route
0	1	4	0	25	4/7/2019	10:13	R5
1	2	4	0	50	4/7/2019	10:23	R5
2	3	6	0	25	4/7/2019	10:33	R5

Description du dataset

Le dataset contient des données collectées lors de 209 vols de drone.

Les données ont été enregistrées à une fréquence d'environ 5 Hz.

Les principales variables disponibles dans le dataset sont:

speed : vitesse horizontale programmée du drone (m/s)

altitude : altitude programmée pendant le vol (m)

payload : masse de la charge transportée (g)

wind_speed : vitesse du vent (m/s)

wind_angle : direction du vent (degrés)

battery_voltage : tension de la batterie (V)

battery_current : courant consommé (A)

position_x, position_y, position_z : position du drone

velocity_x, velocity_y, velocity_z : vitesse du drone

orientation_x, orientation_y, orientation_z, orientation_w : orientation du drone

angular_x, angular_y, angular_z : sa vitesse angulaire

linear_acceleration_x, linear_acceleration_y, linear_acceleration_z :
accélération du drone

La variable cible que l'on souhaite prédire est la consommation énergétique du drone.

3.Exploration et préparation des données

Dans cette étape, nous explorons et préparons les données pour l'analyse.

Les opérations suivantes seront réalisées :

- La vérification de la présence de valeurs manquantes afin d'assurer la qualité des données et d'éviter des erreurs lors de l'entraînement des modèles.
- Création d'une nouvelle variable "puissance" à partir de la tension et du courant de la batterie afin d'obtenir une mesure directement liée à la consommation énergétique($\text{puissance} = \text{tension} \times \text{courant}$).
- Suppression des colonnes initiales utilisées pour calculer la puissance
- Nettoyage de la colonne altitude en remplaçant les valeurs sous forme d'intervalle (ex : "20-50-100") par leur moyenne afin de rendre cette variable exploitable par les modèles
- Suppression des variables qui n'ont pas d'influence sur l'énergie afin de simplifier le modèle et d'améliorer ses performances

```
In [9]: # Vérification des valeurs manquantes
missing = flights.isnull().sum()
missing_pct = (missing / len(flights) * 100).round(2)
missing_df = pd.DataFrame({'Manquants': missing, '% du total': missing_pct})
print(missing_df[missing_df['Manquants'] > 0].to_string())
print('\nAucune valeur manquante.' if missing.sum() == 0 else f'Mi
```

Empty DataFrame
Columns: [Manquants, % du total]
Index: []

Aucune valeur manquante.

```
In [10]: #Création de la colonne puissance
flights["puissance"] = flights['battery_voltage']*flights['battery_
```

```
In [11]: #Suppression des deux colonnes battery_voltage et battery_current
flights = flights.drop(columns=["battery_voltage", "battery_current
```

```
In [12]: flights.head(3)
```

```
Out[12]:
```

	flight	time	wind_speed	wind_angle	position_x	position_y	position_
0	1	0.0	0.1	12	-79.782396	40.458047	269.33240
1	1	0.2	0.1	3	-79.782396	40.458047	269.33205
2	1	0.3	0.1	352	-79.782396	40.458047	269.33308

3 rows x 27 columns

Plus haut dans la première partie, on nous signale que la colonne 24 a un type mixte. En effet, certaines valeurs d'altitude sont fournies sous forme d'intervalle. Nous choisissons de remplacer l'intervalle en la valeur moyenne.

```
In [13]: def moyenne_altitude(val):
            if isinstance(val, str) and "-" in val:
                nums = [float(x) for x in val.split("-")]
                return sum(nums) / len(nums)
            return float(val)

flights["altitude"] = flights["altitude"].apply(moyenne_altitude)
#Affichage des 10 dernières lignes
flights.tail(5)
```

```
Out[13]:
```

	flight	time	wind_speed	wind_angle	position_x	position_y	p
257891	279	152.10	1.1	198	-79.782802	40.459018	27
257892	279	152.20	1.1	196	-79.782802	40.459018	27
257893	279	152.41	1.2	189	-79.782802	40.459018	27
257894	279	152.60	1.1	187	-79.782802	40.459018	27
257895	279	152.70	1.1	184	-79.782802	40.459018	27

5 rows x 27 columns

```
In [14]: #Encodage de l'attribut "Route" à l'aide de la méthode apply.
def route_number(row):
```

```

if row["route"]=="R1":
    return 1
if row["route"]=="R2":
    return 2
if row["route"]=="R3":
    return 3
if row["route"]=="R4":
    return 4
if row["route"]=="R5":
    return 5
if row["route"]=="R6":
    return 6
if row["route"]=="R7":
    return 7
if row["route"]=="A1":
    return 8
if row["route"]=="A2":
    return 9
if row["route"]=="A3":
    return 10
if row["route"]=="H":
    return 11

```

```

flights["route"]=flights.apply(route_number,axis=1)
flights["route"]

```

```

Out[14]: 0      5
         1      5
         2      5
         3      5
         4      5
         ..
        257891    7
        257892    7
        257893    7
        257894    7
        257895    7
        Name: route, Length: 257896, dtype: int64

```

```

In [15]: #supression des colonnes date et time_day
         flights=flights.drop(columns=['date','time_day'])

```

```

In [16]: import numpy as np
         import matplotlib.pyplot as plt

```

```

In [17]: fig, axes = plt.subplots(3, 2, figsize=(18, 18))
         axes = axes.flatten()
         df = flights
         # Histogramme
         axes[0].hist(df['puissance'], bins=80, color='steelblue', edgecolor='black')
         axes[0].axvline(df['puissance'].mean(), color='tomato', linestyle='solid',
                        label=f'Moyenne = {df["puissance"].mean():.0f} W')
         axes[0].axvline(df['puissance'].median(), color='orange', linestyle='solid',
                        label=f'Médiane = {df["puissance"].median():.0f} W')
         axes[0].set_title('Distribution de la puissance')
         axes[0].set_xlabel('Puissance (W)')

```

```

axes[0].set_ylabel('Fréquence')
axes[0].legend()

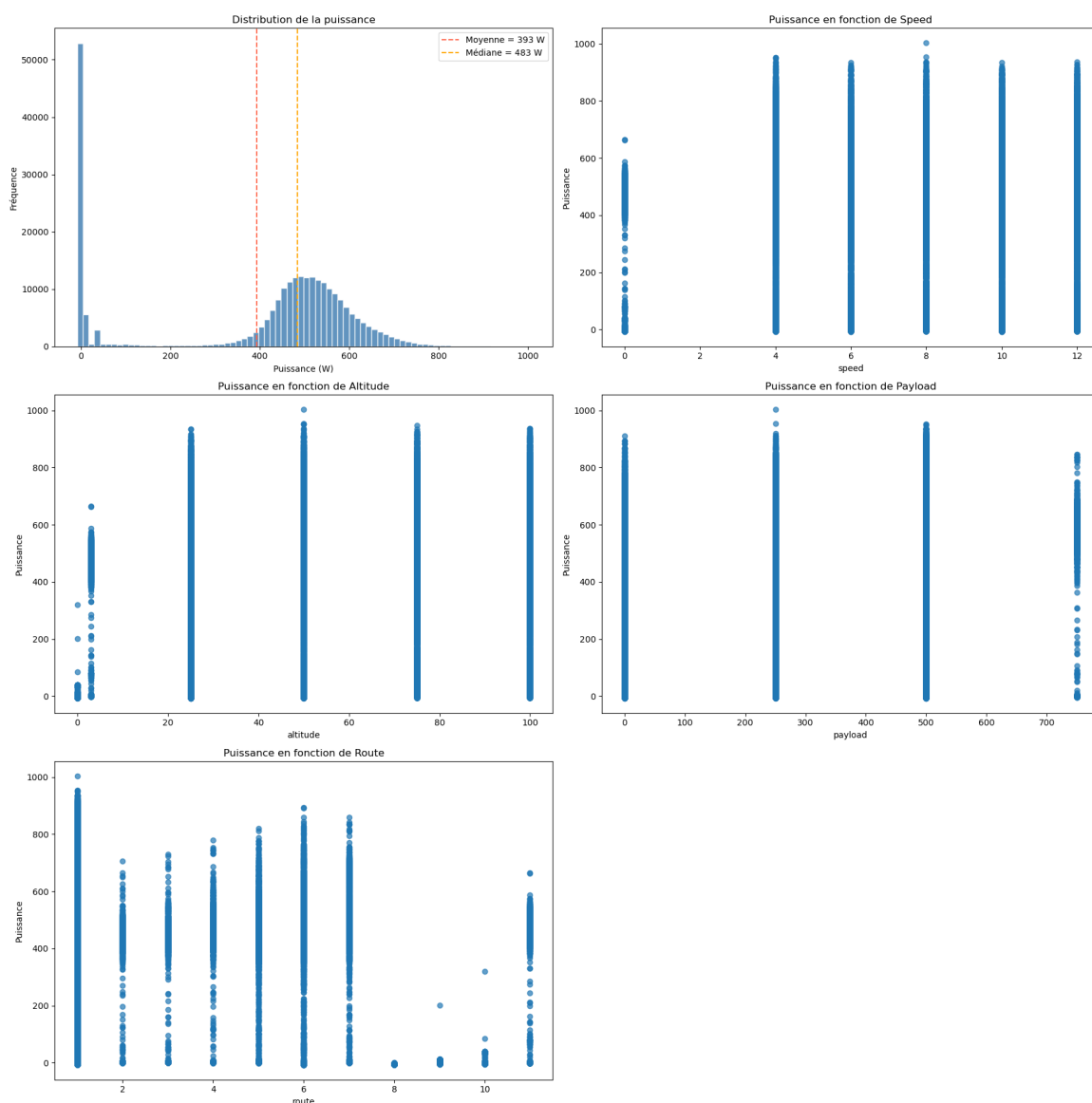
# Scatter plots
plots = [
    ("speed", "Puissance en fonction de Speed"),
    ("altitude", "Puissance en fonction de Altitude"),
    ("payload", "Puissance en fonction de Payload"),
    ("route", "Puissance en fonction de Route"),
]

for i, (col, title) in enumerate(plots, start=1):
    axes[i].scatter(df[col], df["puissance"], alpha=0.7)
    axes[i].set_title(title)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Puissance")

#Supprimer le dernier subplot vide
fig.delaxes(axes[5])

plt.tight_layout()
plt.show()

```



L'histogramme montre la répartition de la puissance et permet de savoir la plage de valeurs qu'elle peut prendre, son ordre de grandeur et la fréquence de chaque valeur dans les données. Une puissance proche de zéro sont prises beaucoup de fois

Les graphes montrent que la puissance ne dépend pas que de la variable globale choisie mais dépend aussi d'autres variables.

Pour une altitude ou vitesse ou payload donné, il existe plusieurs valeurs de puissance .

Des graphiques de type nuage de points seront réalisés en prenant la puissance comme variable en abscisse et les autres variables en ordonnée.

Cela permet d'identifier les tendances, les corrélations potentielles ainsi que la présence de valeurs aberrantes.

Pour une bonne visualisation, nous fixerons une valeur de variable globale.

Remark: Les analyses réalisées sur un vol ou une altitude spécifique ont pour objectif d'illustrer certains comportements. Toutefois, les conclusions du modèle seront basées sur l'ensemble du dataset afin d'assurer une bonne généralisation.

Nous allons prendre le vol 1 pour voir s'il y'a une corrélation entre la puissance et les autres variables

```
In [18]: flights_1=flights[flights["flight"]==1]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)
ax0.set_title("puissance en fonction de velocity_x")
ax0.scatter(flights_1["velocity_x"],flights_1["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("velocity_x")

ax1.set_title("puissance en fonction de velocity_y")
ax1.scatter(flights_1["velocity_y"],flights_1["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("velocity_y")

ax2.set_title("puissance en fonction de velocity_z")
ax2.scatter(flights_1["velocity_z"],flights_1["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("velocity_z")

ax3.set_title("puissance en fonction de angular_x")
ax3.scatter(flights_1["angular_x"],flights_1["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("angular_x")

ax4.set_title("puissance en fonction de angular_y")
ax4.scatter(flights_1["angular_y"],flights_1["puissance"])
```



```
ax4.set_ylabel("puissance")
ax4.set_xlabel("angular_y")

ax5.set_title("puissance en fonction de angular_z")
ax5.scatter(flights_1["angular_z"], flights_1["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("angular_z")

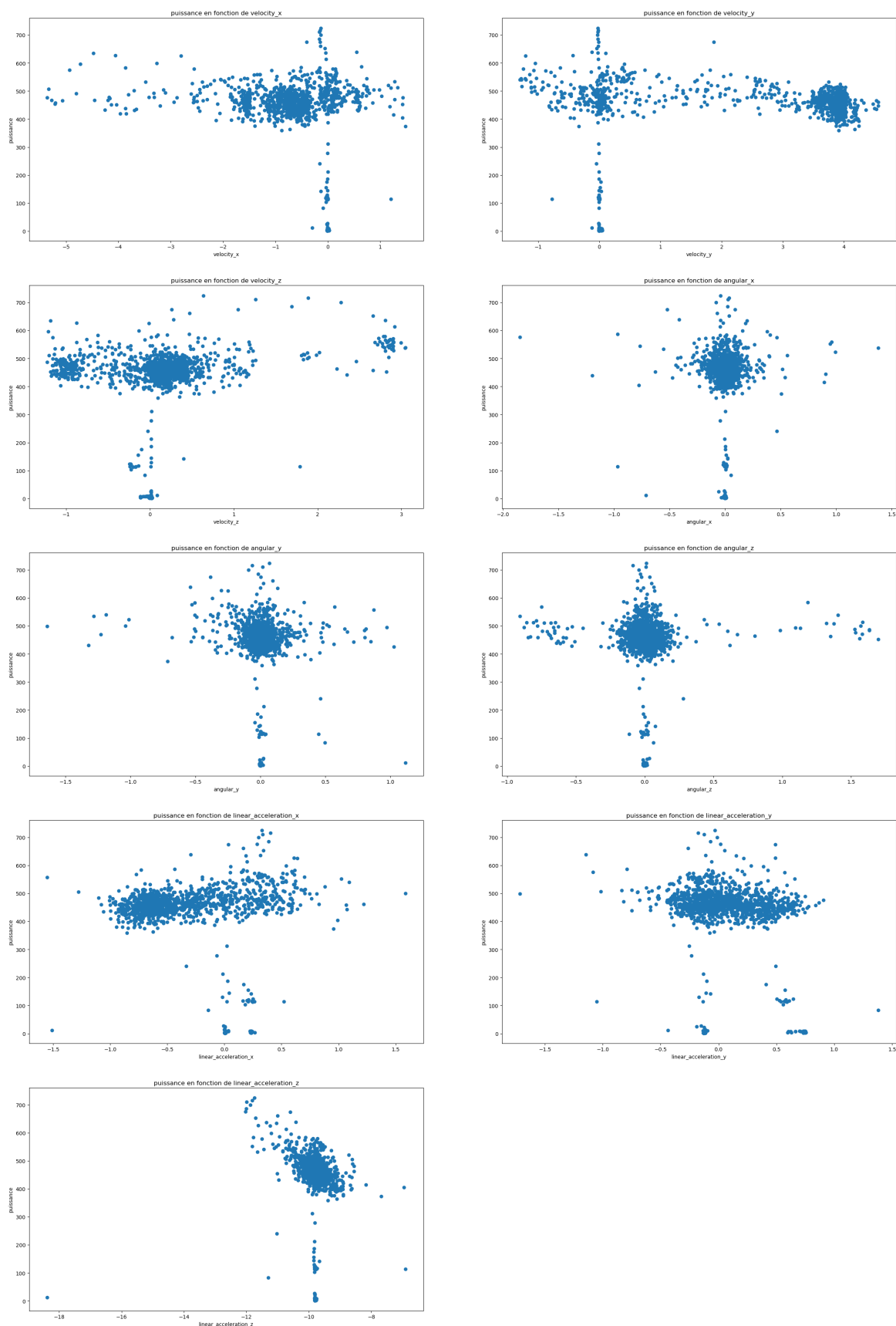
ax6.set_title("puissance en fonction de linear_acceleration_x")
ax6.scatter(flights_1["linear_acceleration_x"], flights_1["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("linear_acceleration_x")

ax7.set_title("puissance en fonction de linear_acceleration_y")
ax7.scatter(flights_1["linear_acceleration_y"], flights_1["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("linear_acceleration_y")

ax8.set_title("puissance en fonction de linear_acceleration_z")
ax8.scatter(flights_1["linear_acceleration_z"], flights_1["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("linear_acceleration_z")

fig.delaxes(ax9)

plt.show()
```



```
In [19]: flights_1=flights[flights["flight"]==1]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)

ax0.set_title("puissance en fonction de time")
ax0.scatter(flights_1["time"],flights_1["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("time")
```

```
ax1.set_title("puissance en fonction de wind_speed")
ax1.scatter(flights_1["wind_speed"], flights_1["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("wind_speed")

ax2.set_title("puissance en fonction de wind_angle")
ax2.scatter(flights_1["wind_angle"], flights_1["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("wind_angle")

ax3.set_title("puissance en fonction de position_x")
ax3.scatter(flights_1["position_x"], flights_1["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("position_x")

ax4.set_title("puissance en fonction de position_y")
ax4.scatter(flights_1["position_y"], flights_1["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("position_y")

ax5.set_title("puissance en fonction de position_z")
ax5.scatter(flights_1["position_z"], flights_1["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("position_z")

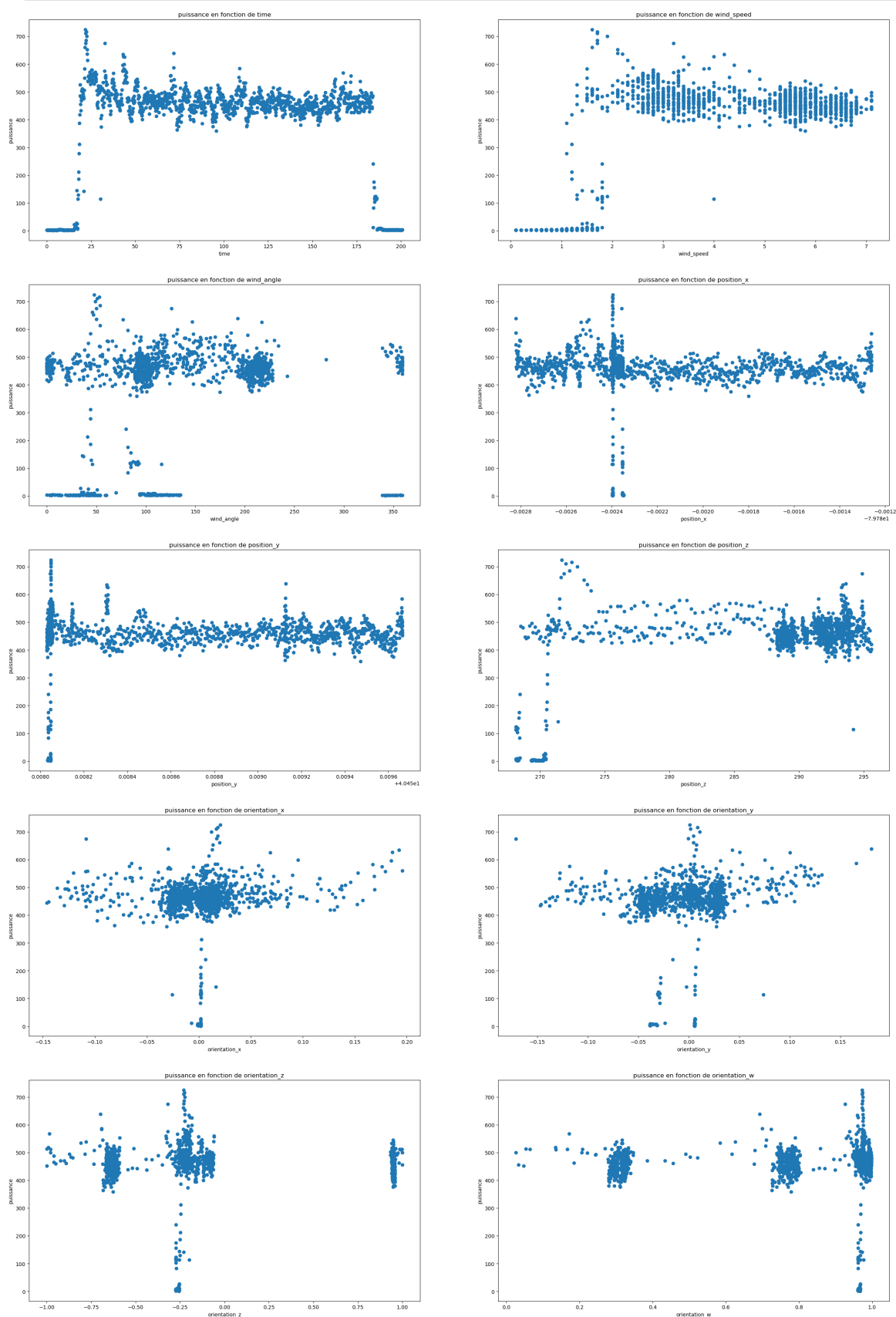
ax6.set_title("puissance en fonction de orientation_x")
ax6.scatter(flights_1["orientation_x"], flights_1["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("orientation_x")

ax7.set_title("puissance en fonction de orientation_y")
ax7.scatter(flights_1["orientation_y"], flights_1["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("orientation_y")

ax8.set_title("puissance en fonction de orientation_z")
ax8.scatter(flights_1["orientation_z"], flights_1["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("orientation_z")

ax9.set_title("puissance en fonction de orientation_w")
ax9.scatter(flights_1["orientation_w"], flights_1["puissance"])
ax9.set_ylabel("puissance")
ax9.set_xlabel("orientation_w")
```

```
plt.show()
```



Nous allons prendre l'altitude 50 par exemple pour voir s'il y'a une corrélation entre la puissance et les autres variables

```
In [20]: flights_altitude=flights[flights["altitude"]==50]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)
ax0.set_title("puissance en fonction de velocity_x")
```

```
ax0.scatter(flights_altitude["velocity_x"], flights_altitude["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("velocity")

ax1.set_title("puissance en fonction de velocity_y")
ax1.scatter(flights_altitude["velocity_y"], flights_altitude["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("velocity_y")

ax2.set_title("puissance en fonction de velocity_z")
ax2.scatter(flights_altitude["velocity_z"], flights_altitude["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("velocity_z")

ax3.set_title("puissance en fonction de angular_x")
ax3.scatter(flights_altitude["angular_x"], flights_altitude["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("angular_x")

ax4.set_title("puissance en fonction de angular_y")
ax4.scatter(flights_altitude["angular_y"], flights_altitude["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("angular_y")

ax5.set_title("puissance en fonction de angular_z")
ax5.scatter(flights_altitude["angular_z"], flights_altitude["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("angular_z")

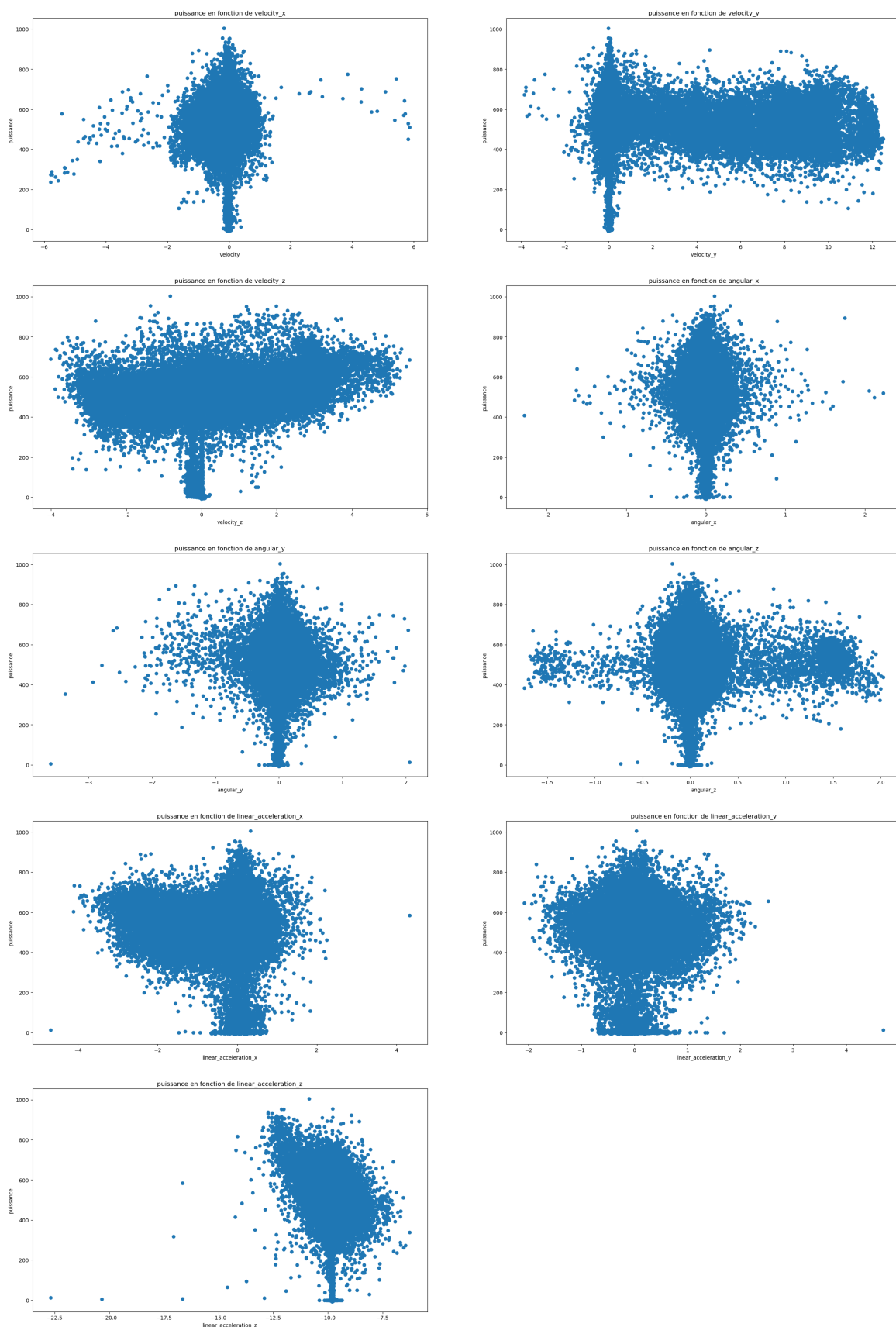
ax6.set_title("puissance en fonction de linear_acceleration_x")
ax6.scatter(flights_altitude["linear_acceleration_x"], flights_altitude["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("linear_acceleration_x")

ax7.set_title("puissance en fonction de linear_acceleration_y")
ax7.scatter(flights_altitude["linear_acceleration_y"], flights_altitude["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("linear_acceleration_y")

ax8.set_title("puissance en fonction de linear_acceleration_z")
ax8.scatter(flights_altitude["linear_acceleration_z"], flights_altitude["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("linear_acceleration_z")

fig.delaxes(ax9)

plt.show()
```



```
In [21]: flights_altitude=flights[flights["altitude"]==50]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)

ax0.set_title("puissance en fonction de time")
ax0.scatter(flights_altitude["time"],flights_altitude["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("time")
```

```
ax1.set_title("puissance en fonction de wind_speed")
ax1.scatter(flights_altitude["wind_speed"], flights_altitude["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("wind_speed")

ax2.set_title("puissance en fonction de wind_angle")
ax2.scatter(flights_altitude["wind_angle"], flights_altitude["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("wind_angle")

ax3.set_title("puissance en fonction de position_x")
ax3.scatter(flights_altitude["position_x"], flights_altitude["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("position_x")

ax4.set_title("puissance en fonction de position_y")
ax4.scatter(flights_altitude["position_y"], flights_altitude["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("position_y")

ax5.set_title("puissance en fonction de position_z")
ax5.scatter(flights_altitude["position_z"], flights_altitude["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("position_z")

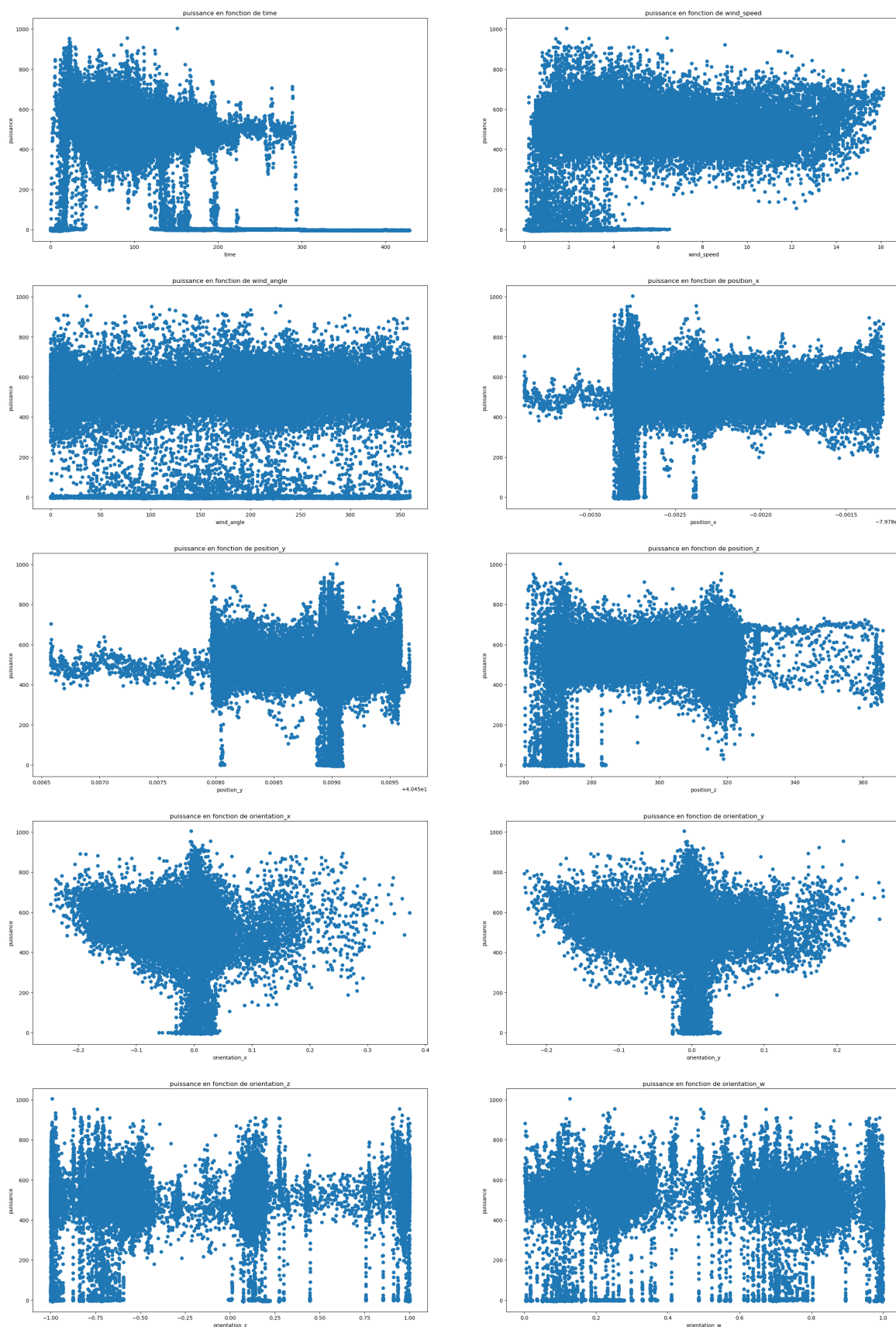
ax6.set_title("puissance en fonction de orientation_x")
ax6.scatter(flights_altitude["orientation_x"], flights_altitude["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("orientation_x")

ax7.set_title("puissance en fonction de orientation_y")
ax7.scatter(flights_altitude["orientation_y"], flights_altitude["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("orientation_y")

ax8.set_title("puissance en fonction de orientation_z")
ax8.scatter(flights_altitude["orientation_z"], flights_altitude["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("orientation_z")

ax9.set_title("puissance en fonction de orientation_w")
ax9.scatter(flights_altitude["orientation_w"], flights_altitude["puissance"])
ax9.set_ylabel("puissance")
ax9.set_xlabel("orientation_w")

plt.show()
```

Nous allons prendre la route H encodée en par 11 par exemple pour voir une s'il y'a une corrélation entre la puissance et les autres variables

```
In [22]: flights_route=flights[flights["route"]==11]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)
ax0.set_title("puissance en fonction de velocity_x")
ax0.scatter(flights_route["velocity_x"],flights_route["puissance"])
```



```
ax0.set_ylabel("puissance")
ax0.set_xlabel("velocity")

ax1.set_title("puissance en fonction de velocity_y")
ax1.scatter(flights_route["velocity_y"], flights_route["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("velocity_y")

ax2.set_title("puissance en fonction de velocity_z")
ax2.scatter(flights_route["velocity_z"], flights_route["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("velocity_z")

ax3.set_title("puissance en fonction de angular_x")
ax3.scatter(flights_route["angular_x"], flights_route["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("angular_x")

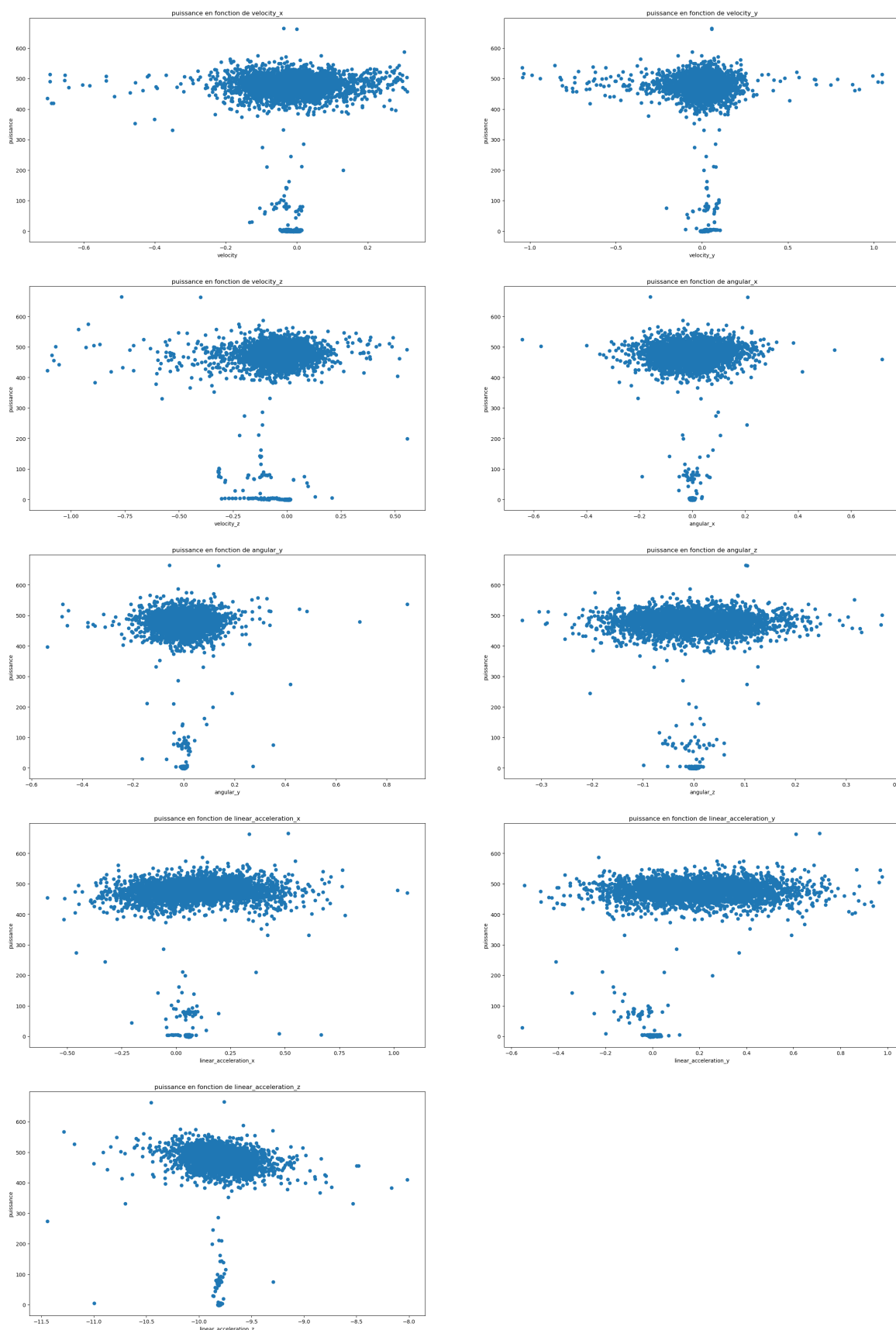
ax4.set_title("puissance en fonction de angular_y")
ax4.scatter(flights_route["angular_y"], flights_route["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("angular_y")

ax5.set_title("puissance en fonction de angular_z")
ax5.scatter(flights_route["angular_z"], flights_route["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("angular_z")

ax6.set_title("puissance en fonction de linear_acceleration_x")
ax6.scatter(flights_route["linear_acceleration_x"], flights_route["p
ax6.set_ylabel("puissance")
ax6.set_xlabel("linear_acceleration_x")

ax7.set_title("puissance en fonction de linear_acceleration_y")
ax7.scatter(flights_route["linear_acceleration_y"], flights_route["p
ax7.set_ylabel("puissance")
ax7.set_xlabel("linear_acceleration_y")

ax8.set_title("puissance en fonction de linear_acceleration_z")
ax8.scatter(flights_route["linear_acceleration_z"], flights_route["p
ax8.set_ylabel("puissance")
ax8.set_xlabel("linear_acceleration_z")
fig.delaxes(ax9)
plt.show()
```



```
In [23]: flights_route=flights[flights["route"]==11]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)

ax0.set_title("puissance en fonction de time")
ax0.scatter(flights_route["time"],flights_route["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("time")
```

```
ax1.set_title("puissance en fonction de wind_speed")
ax1.scatter(flights_route["wind_speed"],flights_route["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("wind_speed")

ax2.set_title("puissance en fonction de wind_angle")
ax2.scatter(flights_route["wind_angle"],flights_route["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("wind_angle")

ax3.set_title("puissance en fonction de position_x")
ax3.scatter(flights_route["position_x"],flights_route["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("position_x")

ax4.set_title("puissance en fonction de position_y")
ax4.scatter(flights_route["position_y"],flights_route["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("position_y")

ax5.set_title("puissance en fonction de position_z")
ax5.scatter(flights_route["position_z"],flights_route["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("position_z")

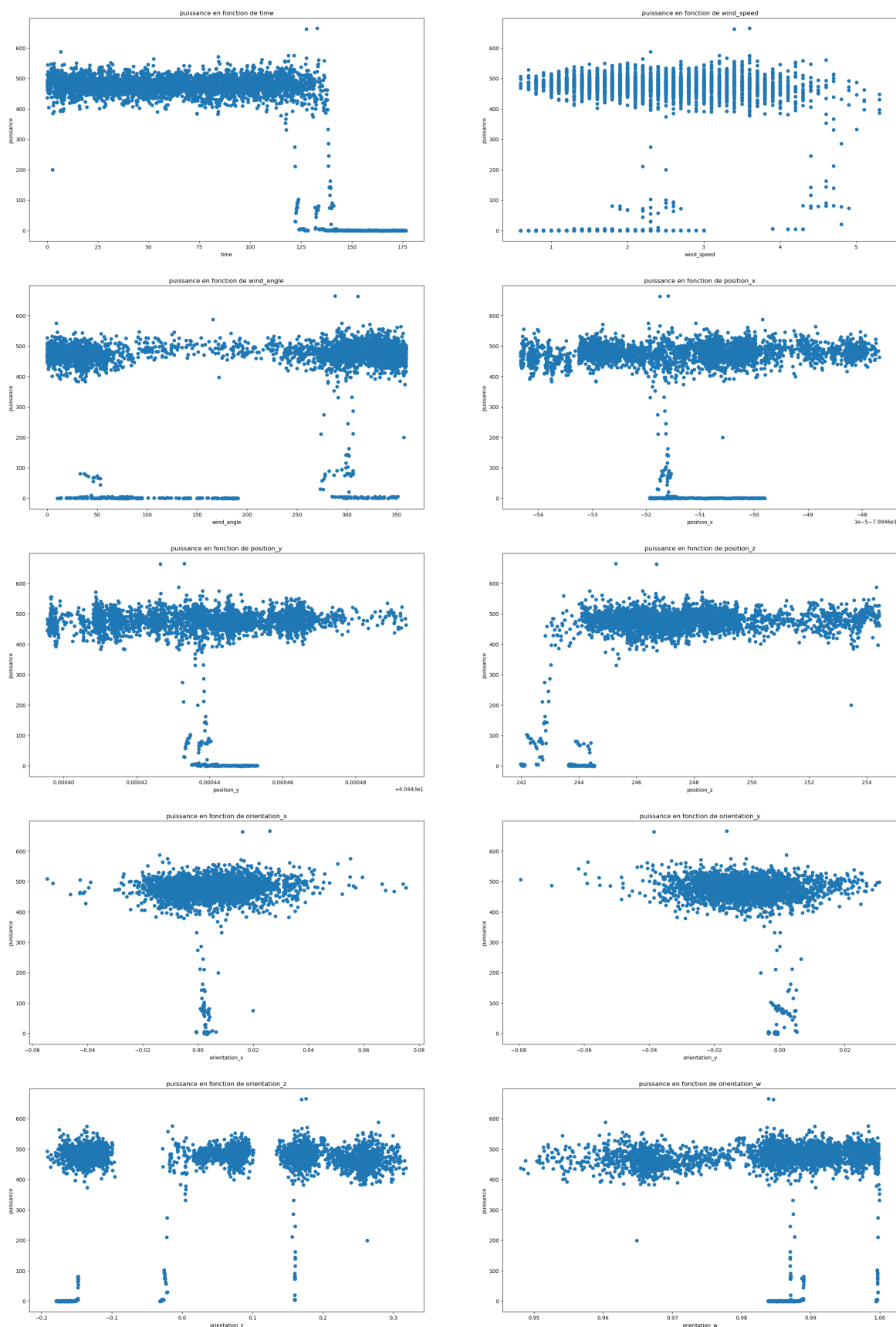
ax6.set_title("puissance en fonction de orientation_x")
ax6.scatter(flights_route["orientation_x"],flights_route["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("orientation_x")

ax7.set_title("puissance en fonction de orientation_y")
ax7.scatter(flights_route["orientation_y"],flights_route["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("orientation_y")

ax8.set_title("puissance en fonction de orientation_z")
ax8.scatter(flights_route["orientation_z"],flights_route["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("orientation_z")

ax9.set_title("puissance en fonction de orientation_w")
ax9.scatter(flights_route["orientation_w"],flights_route["puissance"])
ax9.set_ylabel("puissance")
ax9.set_xlabel("orientation_w")

plt.show()
```



Nous allons prendre une charge de 250 par exemple pour voir une s'il y'a une corrélation entre la puissance et les autres variables

```
In [24]: flights_payload=flights[flights["payload"]==250]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)
ax0.set_title("puissance en fonction de velocity_x")
ax0.scatter(flights_payload["velocity_x"],flights_payload["puissanc
```

```
ax0.set_ylabel("puissance")
ax0.set_xlabel("velocity")

ax1.set_title("puissance en fonction de velocity_y")
ax1.scatter(flights_payload["velocity_y"], flights_payload["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("velocity_y")

ax2.set_title("puissance en fonction de velocity_z")
ax2.scatter(flights_payload["velocity_z"], flights_payload["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("velocity_z")

ax3.set_title("puissance en fonction de angular_x")
ax3.scatter(flights_payload["angular_x"], flights_payload["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("angular_x")

ax4.set_title("puissance en fonction de angular_y")
ax4.scatter(flights_payload["angular_y"], flights_payload["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("angular_y")

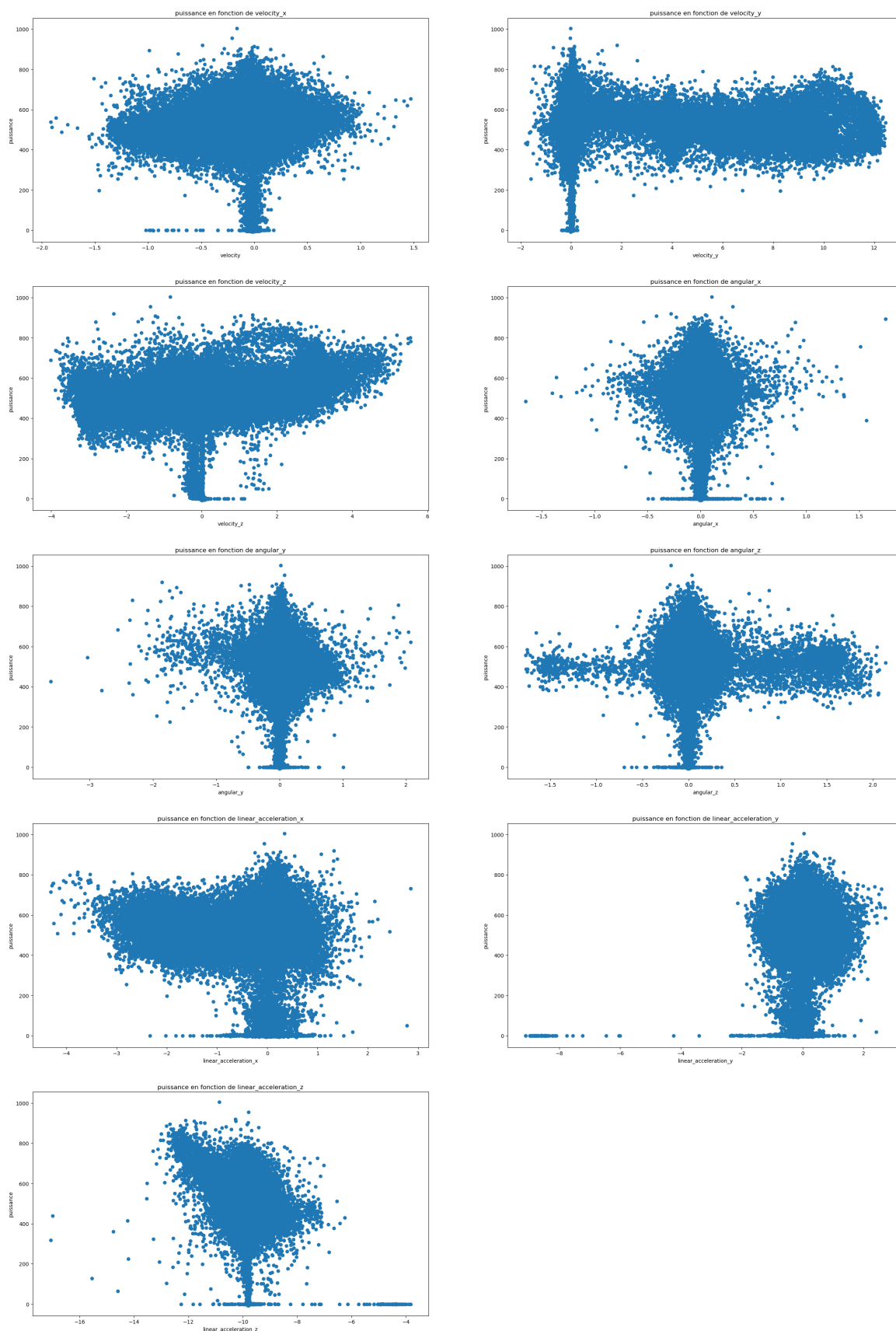
ax5.set_title("puissance en fonction de angular_z")
ax5.scatter(flights_payload["angular_z"], flights_payload["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("angular_z")

ax6.set_title("puissance en fonction de linear_acceleration_x")
ax6.scatter(flights_payload["linear_acceleration_x"], flights_payload["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("linear_acceleration_x")

ax7.set_title("puissance en fonction de linear_acceleration_y")
ax7.scatter(flights_payload["linear_acceleration_y"], flights_payload["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("linear_acceleration_y")

ax8.set_title("puissance en fonction de linear_acceleration_z")
ax8.scatter(flights_payload["linear_acceleration_z"], flights_payload["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("linear_acceleration_z")

fig.delaxes(ax9)
plt.show()
```



```
In [25]: flights_payload=flights[flights["payload"]==250]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)

ax0.set_title("puissance en fonction de time")
ax0.scatter(flights_payload["time"],flights_payload["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("time")
```

```
ax1.set_title("puissance en fonction de wind_speed")
ax1.scatter(flights_payload["wind_speed"], flights_payload["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("wind_speed")

ax2.set_title("puissance en fonction de wind_angle")
ax2.scatter(flights_payload["wind_angle"], flights_payload["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("wind_angle")

ax3.set_title("puissance en fonction de position_x")
ax3.scatter(flights_payload["position_x"], flights_payload["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("position_x")

ax4.set_title("puissance en fonction de position_y")
ax4.scatter(flights_payload["position_y"], flights_payload["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("position_y")

ax5.set_title("puissance en fonction de position_z")
ax5.scatter(flights_payload["position_z"], flights_payload["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("position_z")

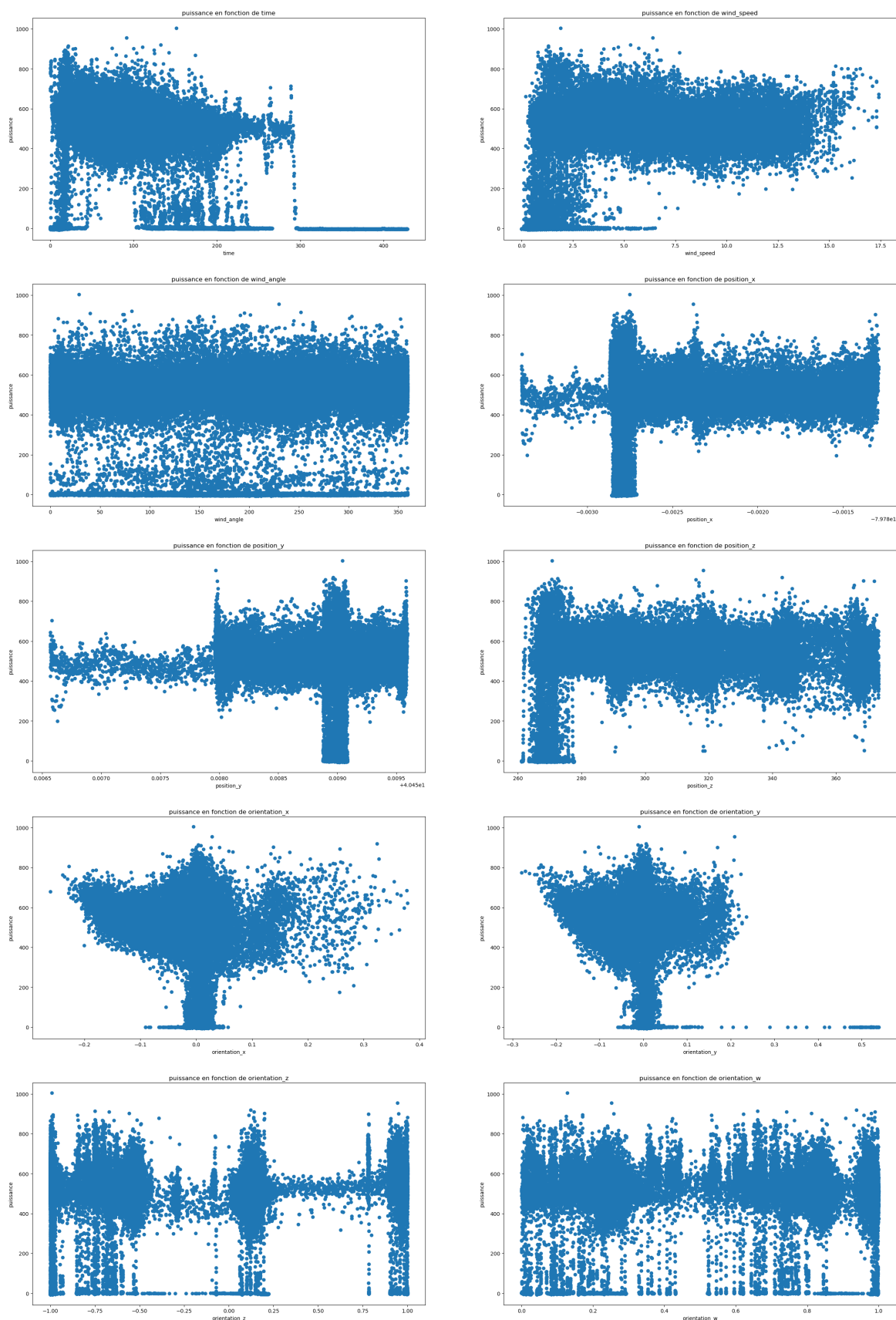
ax6.set_title("puissance en fonction de orientation_x")
ax6.scatter(flights_payload["orientation_x"], flights_payload["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("orientation_x")

ax7.set_title("puissance en fonction de orientation_y")
ax7.scatter(flights_payload["orientation_y"], flights_payload["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("orientation_y")

ax8.set_title("puissance en fonction de orientation_z")
ax8.scatter(flights_payload["orientation_z"], flights_payload["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("orientation_z")

ax9.set_title("puissance en fonction de orientation_w")
ax9.scatter(flights_payload["orientation_w"], flights_payload["puissance"])
ax9.set_ylabel("puissance")
ax9.set_xlabel("orientation_w")

plt.show()
```

Nous allons prendre la vitesse de 12 par exemple pour voir une s'il y'a une corrélation entre la puissance et les autres variables

```
In [26]: flights_speed=flights[flights["speed"]==12]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)
ax0.set_title("puissance en fonction de velocity_x")
ax0.scatter(flights_speed["velocity_x"],flights_speed["puissance"])
```



```
ax0.set_ylabel("puissance")
ax0.set_xlabel("velocity")

ax1.set_title("puissance en fonction de velocity_y")
ax1.scatter(flights_speed["velocity_y"], flights_speed["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("velocity_y")

ax2.set_title("puissance en fonction de velocity_z")
ax2.scatter(flights_speed["velocity_z"], flights_speed["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("velocity_z")

ax3.set_title("puissance en fonction de angular_x")
ax3.scatter(flights_speed["angular_x"], flights_speed["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("angular_x")

ax4.set_title("puissance en fonction de angular_y")
ax4.scatter(flights_speed["angular_y"], flights_speed["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("angular_y")

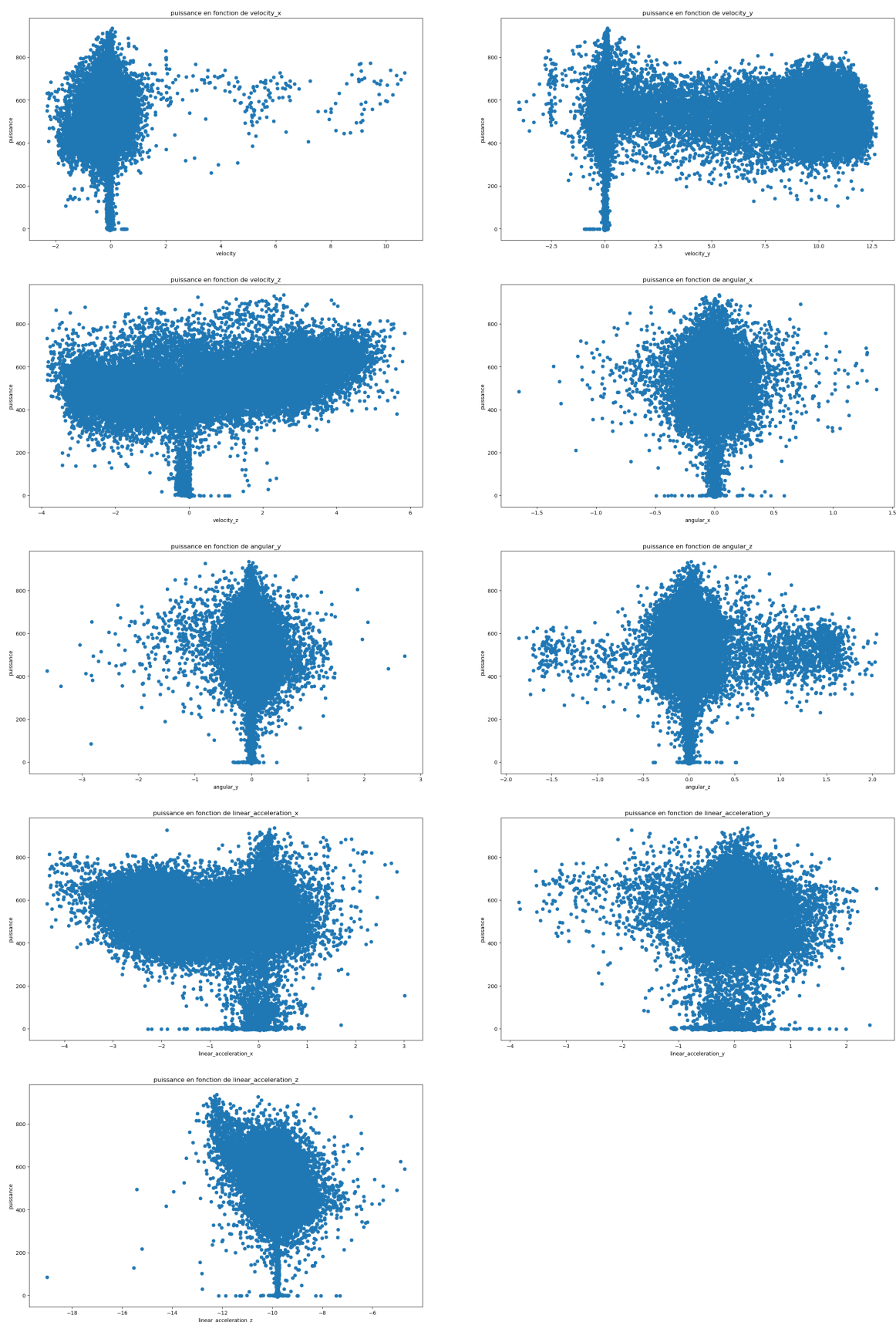
ax5.set_title("puissance en fonction de angular_z")
ax5.scatter(flights_speed["angular_z"], flights_speed["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("angular_z")

ax6.set_title("puissance en fonction de linear_acceleration_x")
ax6.scatter(flights_speed["linear_acceleration_x"], flights_speed["p
ax6.set_ylabel("puissance")
ax6.set_xlabel("linear_acceleration_x")

ax7.set_title("puissance en fonction de linear_acceleration_y")
ax7.scatter(flights_speed["linear_acceleration_y"], flights_speed["p
ax7.set_ylabel("puissance")
ax7.set_xlabel("linear_acceleration_y")

ax8.set_title("puissance en fonction de linear_acceleration_z")
ax8.scatter(flights_speed["linear_acceleration_z"], flights_speed["p
ax8.set_ylabel("puissance")
ax8.set_xlabel("linear_acceleration_z")

fig.delaxes(ax9)
plt.show()
```



```
In [27]: flights_speed=flights[flights["speed"]==12]
fig, ((ax0,ax1),(ax2,ax3),(ax4,ax5),(ax6,ax7),(ax8,ax9)) = plt.subplots(5,2)

ax0.set_title("puissance en fonction de time")
ax0.scatter(flights_speed["time"],flights_speed["puissance"])
ax0.set_ylabel("puissance")
ax0.set_xlabel("time")
```

```
ax1.set_title("puissance en fonction de wind_speed")
ax1.scatter(flights_speed["wind_speed"], flights_speed["puissance"])
ax1.set_ylabel("puissance")
ax1.set_xlabel("wind_speed")

ax2.set_title("puissance en fonction de wind_angle")
ax2.scatter(flights_speed["wind_angle"], flights_speed["puissance"])
ax2.set_ylabel("puissance")
ax2.set_xlabel("wind_angle")

ax3.set_title("puissance en fonction de position_x")
ax3.scatter(flights_speed["position_x"], flights_speed["puissance"])
ax3.set_ylabel("puissance")
ax3.set_xlabel("position_x")

ax4.set_title("puissance en fonction de position_y")
ax4.scatter(flights_speed["position_y"], flights_speed["puissance"])
ax4.set_ylabel("puissance")
ax4.set_xlabel("position_y")

ax5.set_title("puissance en fonction de position_z")
ax5.scatter(flights_speed["position_z"], flights_speed["puissance"])
ax5.set_ylabel("puissance")
ax5.set_xlabel("position_z")

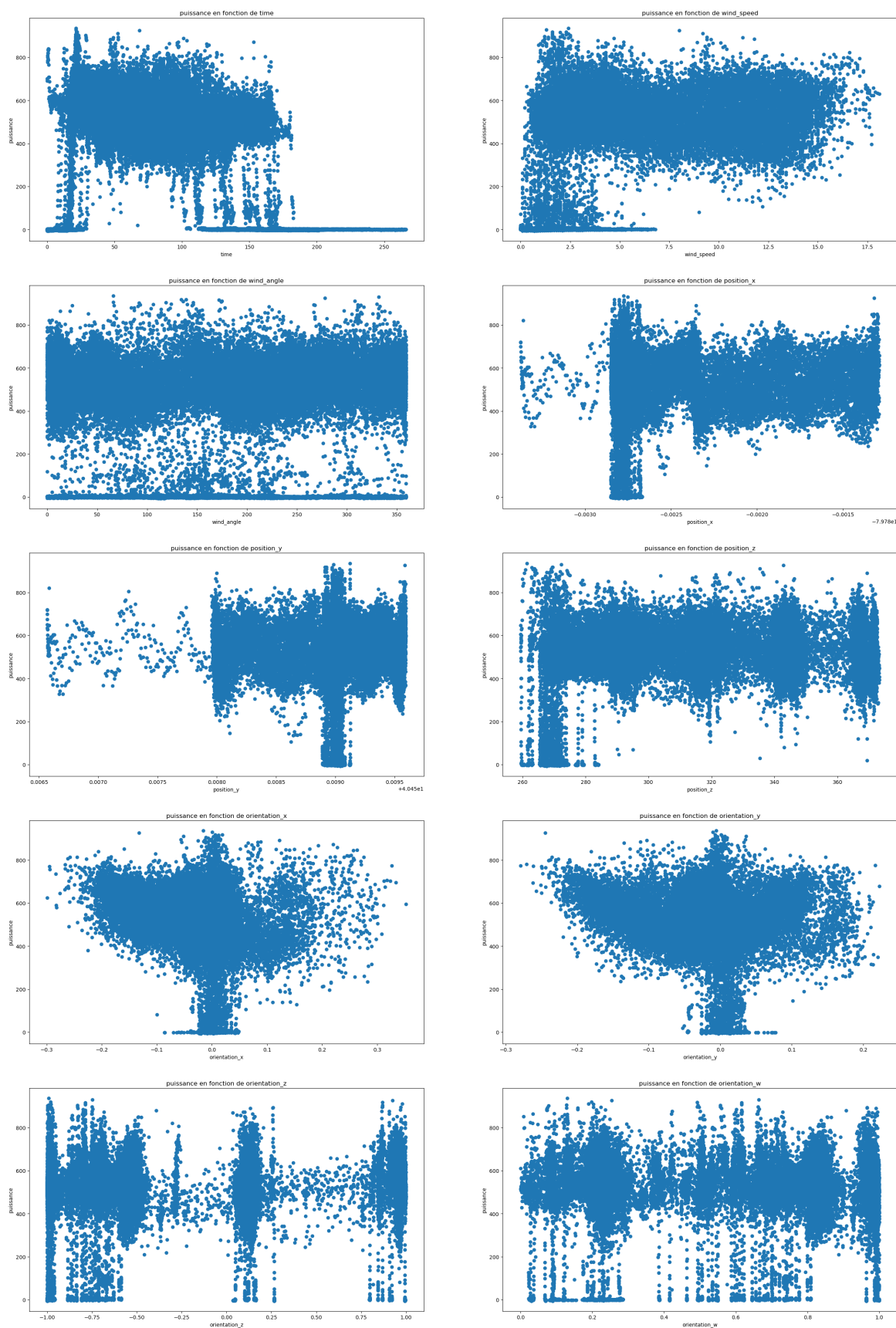
ax6.set_title("puissance en fonction de orientation_x")
ax6.scatter(flights_speed["orientation_x"], flights_speed["puissance"])
ax6.set_ylabel("puissance")
ax6.set_xlabel("orientation_x")

ax7.set_title("puissance en fonction de orientation_y")
ax7.scatter(flights_speed["orientation_y"], flights_speed["puissance"])
ax7.set_ylabel("puissance")
ax7.set_xlabel("orientation_y")

ax8.set_title("puissance en fonction de orientation_z")
ax8.scatter(flights_speed["orientation_z"], flights_speed["puissance"])
ax8.set_ylabel("puissance")
ax8.set_xlabel("orientation_z")

ax9.set_title("puissance en fonction de orientation_w")
ax9.scatter(flights_speed["orientation_w"], flights_speed["puissance"])
ax9.set_ylabel("puissance")
ax9.set_xlabel("orientation_w")

plt.show()
```



La variable "Route" n'apporte pas d'information physique directe pour la prédiction de la consommation énergétique, contrairement à d'autres variables comme la vitesse ou l'altitude.

Sa présence pourrait augmenter la complexité du modèle et entraîner un risque de surapprentissage. De meme que "flight" et "time".

Les composantes de vitesse (velocity_x, velocity_y, velocity_z) ont été combinées pour créer une variable représentant la norme de la vitesse.

Cette transformation permet de simplifier le modèle et de capturer l'intensité globale du mouvement du drone, qui est directement liée à la consommation énergétique.

Les variables initiales ont ensuite été supprimées afin de réduire la dimension du problème et d'éviter la redondance.

```
In [28]: #Création de la colonne velocity
import numpy as np
flights["velocity"] = np.sqrt(flights['velocity_x']*flights['velocity_y']**2+flights['velocity_z']**2)
flights = flights.drop(columns=["velocity_x", "velocity_y", "velocity_z"])
```

```
In [29]: flights.head(3)
```

```
Out[29]:
```

	wind_speed	wind_angle	position_x	position_y	position_z	orientation
0	0.1	12	-79.782396	40.458047	269.332402	0.0017
1	0.1	3	-79.782396	40.458047	269.332056	0.0017
2	0.1	352	-79.782396	40.458047	269.333081	0.0017

Nous allons retenir ces variables comme étant les plus pertinentes:

wind_speed, wind_angle, orientation_x, orientation_y, orientation_z, orientation_w,
velocity, angular_x, angular_y, angular_z, linear_acceleration_x, linear_acceleration_y,
speed, payload, altitude, position_x, position_y, position_z

Avant d'entraîner les modèles, les données sont préparées de la manière suivante :

- Séparation du dataset en jeu d'entraînement et de test 70% d'entraînement et 30% de test.
- Normalisation des variables

```
In [30]: #Séparation des données
from sklearn.model_selection import train_test_split

input_names = ['wind_speed', 'wind_angle', 'orientation_x', 'orientation_y', 'orientation_z', 'orientation_w', 'velocity', 'angular_x', 'angular_y', 'angular_z', 'linear_acceleration_x', 'linear_acceleration_y', 'linear_acceleration_z', 'speed', 'payload', 'altitude', 'position_x', 'position_y', 'position_z']
output_names = ['puissance']

X_train, X_test, y_train, y_test = train_test_split(
    flights[input_names],
    flights[output_names],
    test_size=0.3
)
```

Normalisation des données

Les variables du dataset présentent des échelles différentes (vitesse, accélération, angles, etc.).

Afin d'éviter que certaines variables dominent l'apprentissage du modèle, une normalisation a été appliquée à l'aide de StandardScaler.

Cette méthode permet de centrer les données (moyenne nulle) et de réduire leur variance (écart-type égal à 1), ce qui est particulièrement important pour les modèles sensibles à l'échelle des données tels que les réseaux de neurones et les modèles de type SVR.

```
In [31]: #Normalisation des données
from sklearn import preprocessing

scaler= preprocessing.StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4.Définition du problème d'apprentissage

Le problème est formulé comme un problème d'apprentissage supervisé de type régression.

- Variables d'entrée : paramètres de vol tels que wind_speed,wind_angle,orientation_x,orientation_y,orientation_z,orientation_velocity,angular_x,angular_y,angular_z,linear_acceleration_x,linear_acceleration_y,linear_acceleration_z,payload,altitude,position_x,position_y,position_z.
- Variable de sortie : puissance

L'objectif est d'apprendre une fonction permettant de prédire la puissance donc la consommation énergétique à partir des variables d'entrée.

5.Entraînement et comparaison de modèles

Model selection

La sélection du modèle se fera selon les étapes suivantes :

- Entraîner plusieurs modèles sur les données d'entraînement
- Évaluer leurs performances sur un jeu de validation
- Comparer les résultats selon différentes métriques
- Sélectionner le modèle offrant les meilleures performances

Évaluation des performances

Les modèles seront évalués à l'aide des métriques suivantes :

- R squared
- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)

Ces indicateurs nous permettront de mesurer la précision des prédictions. Le RMSE sera retenu comme métrique principale pour la sélection du modèle final, car il permet de mesurer directement l'erreur de prédiction en unités réelles.

Conception des modèles

Nous allons utiliser plusieurs modèles de machine learning pour prédire la consommation énergétique :

- Régression linéaire
- Régression polynomiale
- les réseaux de neurones
- Random Forest
- SVR

modèle 1 : Linear Regression

Principe: La régression linéaire consiste à modéliser la relation entre les variables d'entrée et la variable cible comme une combinaison linéaire des variables explicatives.

<https://docs.kanaries.net/fr/topics/Scikit-Learn/sklearn-linear-regression>

```
In [32]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error
import time

# Train model
model_1 = LinearRegression()
start = time.time()
model_1.fit(X_train, y_train)
train_time = time.time() - start
y_pred_1 = model_1.predict(X_test)

r2 = r2_score(y_test, y_pred_1)
mse = mean_squared_error(y_test, y_pred_1)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred_1)
print(f'Temps d'entraînement: {train_time:.2f} secondes")
print(f'R² Score: {r2:.4f}")
```



```
print(f"MSE:      {mse:.4f}")
print(f"RMSE:     {rmse:.4f}")
print(f"MAE:      {mae:.4f}")
test_score = model_1.score(X_test, y_test)
```

Temps d'entraînement: 0.13 secondes

R² Score: 0.5264

MSE: 26502.5172

RMSE: 162.7959

MAE: 133.9525

Ce modele n'est pas adapté. La relation entre la puissance et les autres variables n'est donc pas linéaire

Modèle 2: Régression polynomiale

Quand la relation entre variables et cible n'est pas linéaire, les variables polynomiales peuvent capturer les courbes et les interactions

La régression polynomiale permettra donc de mieux représenter ces relations physiques non linéaires.

Principe: La régression polynomiale est une extension de la régression linéaire qui permet de modéliser des relations non linéaires entre les variables d'entrée et la variable cible en introduisant des termes de degré supérieur (carrés, cubes, etc.).

```
In [33]: from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
import time
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

# Polynomial (degree 2) model
poly_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('linear', LinearRegression())
])
start_2 = time.time()
poly_pipeline.fit(X_train, y_train)
train_time_2 = time.time() - start_2

# Degree 2
y_pred_2 = poly_pipeline.predict(X_test)

rmse_2 = np.sqrt(mean_squared_error(y_test, y_pred_2))
mae_2 = mean_absolute_error(y_test, y_pred_2)
r2_2 = r2_score(y_test, y_pred_2)
print(f"Temps d'entraînement deg2: {train_time_2:.2f} secondes")
```



```

print(f"Poly (d=2) RMSE: {rmse_2:.4f}")
print(f"Poly (d=2) MAE: {mae_2:.4f}")
print(f"Poly (d=2) R²: {r2_score(y_test, poly_pipeline.predict(X_te

# Polynomial (degree 3) model
poly3_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('linear', LinearRegression())
])
start_3 = time.time()
poly3_pipeline.fit(X_train, y_train)
train_time_3 = time.time() - start_3
# Degree 3
y_pred_3 = poly3_pipeline.predict(X_test)

rmse_3 = np.sqrt(mean_squared_error(y_test, y_pred_3))
mae_3 = mean_absolute_error(y_test, y_pred_3)
print(f"Temps d'entraînement_deg3: {train_time_3:.2f} secondes")
print(f"Poly (d=3) R²: {r2_score(y_test, poly3_pipeline.predict(X_t
print(f"Poly (d=3) RMSE: {rmse_3:.4f}")
print(f"Poly (d=3) MAE: {mae_3:.4f}")

#Polynomial (degree 4) model
poly4_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=4, include_bias=False)),
    ('linear', LinearRegression())
])
start_4= time.time()
poly4_pipeline.fit(X_train, y_train)
train_time_4 = time.time() - start_4
# Degree 4
y_pred_4 = poly4_pipeline.predict(X_test)

rmse_4 = np.sqrt(mean_squared_error(y_test, y_pred_4))
mae_4 = mean_absolute_error(y_test, y_pred_4)
print(f"Poly (d=4) R²: {r2_score(y_test, poly4_pipeline.predict(X_t
print(f"Poly (d=4) RMSE: {rmse_4:.4f}")
print(f"Poly (d=4) MAE: {mae_4:.4f}")

```

```

Temps d'entraînement_deg2: 3.23 secondes
Poly (d=2) RMSE: 108.7573
Poly (d=2) MAE: 80.1061
Poly (d=2) R²: 0.7886
Temps d'entraînement_deg3: 50.52 secondes
Poly (d=3) R²: 0.8812
Poly (d=3) RMSE: 81.5265
Poly (d=3) MAE: 55.7447
Poly (d=4) R²: 0.9036
Poly (d=4) RMSE: 73.4425
Poly (d=4) MAE: 44.4172

```

Modèle 3: Réseaux de neurones

Principe: Le MLPRegressor (Multi-Layer Perceptron) est un modèle de réseau de neurones artificiels composé de plusieurs couches de neurones, capable

d'apprendre des relations complexes et non linéaires entre les variables d'entrée et la variable cible.

Un MLP contient : couche d'entrée → les variables d'entrée

couches cachées → transformations non linéaires

couche de sortie → prédiction de la puissance

Le MLPRegressor permet de modéliser des relations très complexes entre les paramètres de vol et la puissance, notamment lorsque ces relations sont fortement non linéaires et difficiles à exprimer analytiquement.

```
In [34]: import time
scaler_X = preprocessing.MinMaxScaler()
scaler_Y = preprocessing.MinMaxScaler()
X_train3 = scaler_X.fit_transform(X_train)
X_test3 = scaler_X.transform(X_test)
y_train3 = scaler_Y.fit_transform(y_train)
y_test3 = scaler_Y.transform(y_test)

from sklearn.neural_network import MLPRegressor

model_MLP = MLPRegressor(
    hidden_layer_sizes=(128, 64, 32), # 3 couches cachées
    activation='relu',
    solver='adam',
    alpha=0.001, # régularisation L2
    max_iter=1000,
    random_state=42,
    verbose=True,
    learning_rate_init=0.001,
    early_stopping=True, # arrêt si pas d'améliorati
    validation_fraction=0.1
)

start = time.time()
model_MLP.fit(X_train3, y_train3.ravel())
train_time = time.time() - start
# Prédiction et dénormalisation
y_pred_norm = model_MLP.predict(X_test3)
y_pred = scaler_Y.inverse_transform(y_pred_norm[:, np.newaxis])
y_true = scaler_Y.inverse_transform(y_test3)

# Métriques
from sklearn.metrics import mean_squared_error, mean_absolute_error
print(f"Temps d'entraînement: {train_time:.2f} secondes")
print("RMSE:", np.sqrt(mean_squared_error(y_true, y_pred)))
print("MAE:", mean_absolute_error(y_true, y_pred)) # pas de s
print("R²:", r2_score(y_true, y_pred))
print(f"Itérations effectuées : {model_MLP.n_iter_}")
```

Iteration 1, loss = 0.00600723

Validation score: 0.833837

Iteration 2, loss = 0.00342841

Validation score: 0.894296
Iteration 3, loss = 0.00308314
Validation score: 0.898553
Iteration 4, loss = 0.00302529
Validation score: 0.905267
Iteration 5, loss = 0.00295406
Validation score: 0.893286
Iteration 6, loss = 0.00285908
Validation score: 0.877681
Iteration 7, loss = 0.00286997
Validation score: 0.907535
Iteration 8, loss = 0.00277010
Validation score: 0.907584
Iteration 9, loss = 0.00275072
Validation score: 0.913417
Iteration 10, loss = 0.00271071
Validation score: 0.912666
Iteration 11, loss = 0.00269939
Validation score: 0.893509
Iteration 12, loss = 0.00261223
Validation score: 0.900647
Iteration 13, loss = 0.00263466
Validation score: 0.915272
Iteration 14, loss = 0.00259070
Validation score: 0.916954
Iteration 15, loss = 0.00260790
Validation score: 0.917571
Iteration 16, loss = 0.00257025
Validation score: 0.913782
Iteration 17, loss = 0.00253245
Validation score: 0.919763
Iteration 18, loss = 0.00253998
Validation score: 0.920251
Iteration 19, loss = 0.00251401
Validation score: 0.917829
Iteration 20, loss = 0.00247262
Validation score: 0.915808
Iteration 21, loss = 0.00247702
Validation score: 0.917928
Iteration 22, loss = 0.00245805
Validation score: 0.921969
Iteration 23, loss = 0.00244394
Validation score: 0.919922
Iteration 24, loss = 0.00246332
Validation score: 0.922102
Iteration 25, loss = 0.00244580
Validation score: 0.913315
Iteration 26, loss = 0.00240990
Validation score: 0.927649
Iteration 27, loss = 0.00239191
Validation score: 0.921459
Iteration 28, loss = 0.00236999
Validation score: 0.917882
Iteration 29, loss = 0.00242023
Validation score: 0.916134
Iteration 30, loss = 0.00240268

Validation score: 0.926261
Iteration 31, loss = 0.00236100
Validation score: 0.924685
Iteration 32, loss = 0.00236650
Validation score: 0.927263
Iteration 33, loss = 0.00232972
Validation score: 0.921624
Iteration 34, loss = 0.00234347
Validation score: 0.923457
Iteration 35, loss = 0.00232676
Validation score: 0.922193
Iteration 36, loss = 0.00230086
Validation score: 0.930273
Iteration 37, loss = 0.00231845
Validation score: 0.929099
Iteration 38, loss = 0.00228153
Validation score: 0.925509
Iteration 39, loss = 0.00225899
Validation score: 0.930292
Iteration 40, loss = 0.00226790
Validation score: 0.918540
Iteration 41, loss = 0.00228843
Validation score: 0.917244
Iteration 42, loss = 0.00230823
Validation score: 0.929882
Iteration 43, loss = 0.00227809
Validation score: 0.891363
Iteration 44, loss = 0.00225117
Validation score: 0.920660
Iteration 45, loss = 0.00223475
Validation score: 0.931146
Iteration 46, loss = 0.00223812
Validation score: 0.931158
Iteration 47, loss = 0.00221913
Validation score: 0.929912
Iteration 48, loss = 0.00223740
Validation score: 0.927128
Iteration 49, loss = 0.00218908
Validation score: 0.928107
Iteration 50, loss = 0.00222097
Validation score: 0.922689
Iteration 51, loss = 0.00219711
Validation score: 0.927577
Iteration 52, loss = 0.00219407
Validation score: 0.930272
Iteration 53, loss = 0.00219885
Validation score: 0.911825
Iteration 54, loss = 0.00222506
Validation score: 0.932744
Iteration 55, loss = 0.00216121
Validation score: 0.929956
Iteration 56, loss = 0.00219053
Validation score: 0.935340
Iteration 57, loss = 0.00214506
Validation score: 0.931566
Iteration 58, loss = 0.00216803

```

Validation score: 0.926217
Iteration 59, loss = 0.00215224
Validation score: 0.924880
Iteration 60, loss = 0.00217786
Validation score: 0.926858
Iteration 61, loss = 0.00215034
Validation score: 0.930371
Iteration 62, loss = 0.00215340
Validation score: 0.934683
Iteration 63, loss = 0.00214819
Validation score: 0.933171
Iteration 64, loss = 0.00216100
Validation score: 0.934451
Iteration 65, loss = 0.00214985
Validation score: 0.923940
Iteration 66, loss = 0.00215810
Validation score: 0.928232
Iteration 67, loss = 0.00209194
Validation score: 0.928692
Validation score did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
Temps d'entraînement: 87.18 secondes
RMSE: 59.62425604984698
MAE: 35.11262931487603
R²: 0.9364648350050279
Itérations effectuées : 67

```

Modèle 4 : Forêts aléatoires (*Random forest regression*)

Principe: La Random Forest est un ensemble d'arbres de décision entraînés sur des sous-parties aléatoires des données, dont les prédictions sont moyennées.

Elle permet de modéliser des relations complexes et non linéaires entre les variables et la consommation énergétique.

Hyperparamètre: nombre d'estimateur.

ressources: <https://docs.kanaries.net/fr/topics/Scikit-Learn/sklearn-random-forest>

```

In [35]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
import time
# Train regressor
rf_reg = RandomForestRegressor(
    n_estimators=500,
    max_depth=None,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
start = time.time()
rf_reg.fit(X_train, y_train)

```

```

train_time = time.time() - start
y_pred = rf_reg.predict(X_test)

# Evaluation metrics
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
score = rf_reg.score(X_test, y_test)
print(f"Temps d'entraînement: {train_time:.2f} secondes")
print(f"R-squared: {r2:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MAE: {mae:.4f}")
print(f"SCORE: {score:.4f}")

```

C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return fit_method(estimator, *args, **kwargs)
```

Temps d'entraînement: 506.18 secondes

R-squared: 0.9666

RMSE: 43.2504

MAE: 25.5236

SCORE: 0.9666

La validation croisée a été utilisée afin d'obtenir une estimation plus robuste des performances du modèle.

Cette méthode consiste à entraîner et tester le modèle sur différentes partitions des données, ce qui permet de réduire l'impact du hasard dans la séparation des données

```

In [36]: from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor

scores = cross_val_score(rf_reg, X_train, y_train, cv=5, scoring="r2")

print("Scores:", scores)
print("R2 moyen:", scores.mean())

```

```

C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataC
onversionWarning: A column-vector y was passed when a 1d array was e
xpected. Please change the shape of y to (n_samples,), for example u
sing ravel().
    return fit_method(estimator, *args, **kwargs)
C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataC
onversionWarning: A column-vector y was passed when a 1d array was e
xpected. Please change the shape of y to (n_samples,), for example u
sing ravel().
    return fit_method(estimator, *args, **kwargs)
C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataC
onversionWarning: A column-vector y was passed when a 1d array was e
xpected. Please change the shape of y to (n_samples,), for example u
sing ravel().
    return fit_method(estimator, *args, **kwargs)
C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataC
onversionWarning: A column-vector y was passed when a 1d array was e
xpected. Please change the shape of y to (n_samples,), for example u
sing ravel().
    return fit_method(estimator, *args, **kwargs)
C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\base.py:1365: DataC
onversionWarning: A column-vector y was passed when a 1d array was e
xpected. Please change the shape of y to (n_samples,), for example u
sing ravel().
    return fit_method(estimator, *args, **kwargs)
Scores: [0.96362222 0.96404975 0.96421829 0.96588318 0.96490879]
R2 moyen: 0.9645364463400679

```

Modèle 5 : Machine à Vecteur de Support Regression

Le Support Vector Regression (SVR) est une méthode de régression issue des machines à vecteurs de support (SVM).

Principe: L'objectif du SVR est de trouver une fonction qui approxime les données tout en autorisant une marge d'erreur ϵ . Contrairement aux méthodes classiques de régression, le SVR ne cherche pas à minimiser directement l'erreur pour tous les points, mais à maintenir les prédictions dans un intervalle de tolérance autour des valeurs réelles.

Seules certaines observations, appelées vecteurs de support, influencent la construction du modèle. Ces points correspondent aux observations situées en dehors de la zone de tolérance ou sur ses frontières.

L'utilisation d'un noyau (kernel), comme le noyau gaussien (RBF), permet de modéliser des relations non linéaires entre les variables d'entrée et la variable cible.

Le modèle SVR présente une complexité de calcul élevée, notamment lorsque le nombre d'observations est important. En effet, son temps d'entraînement

augmente fortement avec la taille du dataset.

Afin de rendre l'entraînement du modèle tractable en temps de calcul, un sous-échantillon des données d'entraînement a été utilisé.

Cette approche permet de réduire significativement le temps d'entraînement tout en conservant une représentation globale des données, à condition que l'échantillonnage soit réalisé de manière aléatoire.

Le modèle ainsi entraîné peut ensuite être évalué sur l'ensemble du jeu de test afin d'estimer correctement ses performances.

```
In [37]: import time
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import numpy as np

# Sous-échantillonnage pour rendre le SVR tractable
n_sample = 50000
np.random.seed(42)
idx = np.random.choice(len(X_train), size=n_sample, replace=False)
X_train_svr = X_train[idx]
y_train_svr = y_train.iloc[idx]

print(f"Entraînement SVR sur {n_sample} échantillons (au lieu de {len(X_train)})")

# Entraînement du modèle SVR
svr_model = SVR(kernel='rbf', C=100, epsilon=0.1)
start = time.time()
svr_model.fit(X_train_svr, y_train_svr)
train_time = time.time() - start
# Prédiction sur le jeu de test complet
y_pred_svr = svr_model.predict(X_test)

# Évaluation
mse = mean_squared_error(y_test, y_pred_svr)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_svr)
print(f"Temps d'entraînement: {train_time:.2f} secondes")
print("SVR Results:")
print("RMSE:", rmse)
print("R²:", r2)
```

Entraînement SVR sur 50000 échantillons (au lieu de 180527)

C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

y = column_or_1d(y, warn=True)

Temps d'entraînement: 406.63 secondes

SVR Results:

RMSE: 83.94559186159111

R²: 0.8740599079187787

Une recherche d'hyperparamètres sera réalisée à l'aide de GridSearchCV afin d'optimiser les performances du modèle SVR.

Les paramètres C, epsilon et gamma ont été ajustés en utilisant une validation croisée.

Afin de réduire le temps de calcul, cette recherche sera effectuée aussi sur un sous-échantillon des données d'entraînement.

```
In [38]: from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR

param_grid = {
    'C': [1, 10, 100],
    'epsilon': [0.01, 0.1, 1],
    'gamma': ['scale', 0.1],
    'kernel': ['rbf']
}

grid = GridSearchCV(
    SVR(),
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    verbose=1,
    n_jobs=-1
)

# le sous-échantillon
grid.fit(X_train_svr, y_train_svr)

print("Best parameters:", grid.best_params_)

best_svr = grid.best_estimator_

y_pred = best_svr.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Optimisation de SVR:")
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)
```

Fitting 3 folds for each of 18 candidates, totalling 54 fits

```
C:\Users\DEM\anaconda3\Lib\site-packages\sklearn\utils\validation.p
y:1406: DataConversionWarning: A column-vector y was passed when a 1
d array was expected. Please change the shape of y to (n_samples, ),
for example using ravel().
y = column_or_1d(y, warn=True)
```

Best parameters: {'C': 100, 'epsilon': 1, 'gamma': 0.1, 'kernel': 'rbf'}

Optimisation de SVR:

RMSE: 74.05905637005687

MAE: 45.138906491982944

R²: 0.9019777732949242

Le meilleur modèle est le Random Forest car il présente un RMSE plus faible de 43.2504 .

Conclusion

Dans ce projet, plusieurs modèles de régression ont été étudiés afin de prédire la consommation énergétique du drone à partir de variables physiques et environnementales.

Les étapes de préparation des données ont permis d'améliorer la qualité du dataset, notamment grâce au nettoyage des données, à la création de variables pertinentes (comme la vitesse ou la puissance) et à la suppression des variables non informatives.

Différents modèles ont ensuite été testés, tels que la régression linéaire, la régression polynomiale, le SVR, les réseaux de neurones et le Random Forest. Les performances ont été évaluées à l'aide de plusieurs métriques, dont le RMSE, le MAE et le coefficient R², le RMSE ayant été retenu comme critère principal de comparaison.

Les résultats montrent que le modèle Random Forest est le plus performant, avec le RMSE le plus faible. Ce modèle est particulièrement adapté car il permet de capturer des relations non linéaires complexes tout en étant robuste face au bruit et aux interactions entre variables.

En perspective, plusieurs améliorations peuvent être envisagées. Une optimisation plus poussée des hyperparamètres du Random Forest pourrait permettre d'améliorer encore les performances. L'exploration de modèles plus avancés, ainsi qu'une analyse approfondie de l'importance des variables, permettraient d'enrichir l'étude et de mieux comprendre les facteurs influençant la consommation énergétique.